

Capitolo 8 - Software Malevoli

Tipologie di Malware

Il termine malware racchiude diverse categorie di minacce, distinte principalmente per il modo in cui si propagano e per l'azione maligna (payload) che eseguono.

I **Virus** sono programmi in grado di replicarsi e passare il proprio codice infetto ad altri programmi, modificandoli. Non possono essere eseguiti o replicati senza un programma ospite (host) e si dividono in "transient" (la cui vita dipende dall'esecuzione del programma ospite) e "resident" (che si posizionano in memoria e terminano solo allo spegnimento del sistema). I **Worm**, al contrario, sono programmi *standalone* che non necessitano di un host e diffondono copie di se stessi attraverso la rete. I **Trojan Horse** sono invece codici che celano un effetto maligno e non ovvio dietro a un'operazione apparentemente legittima, come uno script di login che intercetta e conserva le credenziali dell'utente.

Esistono molte altre nomenclature per descrivere le azioni specifiche dei malware. Tra questi troviamo le *Logic Bomb* e le *Time Bomb*, che scattano al verificarsi di specifiche condizioni logiche o temporali, i *Dropper*, che fungono da vettori di trasferimento per installare altri malware, e i *Ransomware*, che criptano i dati esfiltrandoli e richiedendo un riscatto. Troviamo inoltre gli *Spyware* che intercettano dati segretamente, i *Bot/Zombie* che trasformano il computer in un agente controllato a distanza, i *Rootkit* che si installano nelle sezioni più privilegiate dell'OS rendendosi quasi invisibili, e le *Trapdoor* (o Backdoor) che aggirano i normali controlli di autenticazione.

I Quattro Aspetti delle Infezioni da Malware

L'analisi delle infezioni da codice maligno si concentra su quattro aspetti fondamentali: il danno causato, i meccanismi di trasmissione e propagazione, l'attivazione del payload e le tecniche per rimanere invisibili.

1. Il Danno (Harm)

I payload del malware possono avere impatti differenti, classificati in tre tipologie.

- I danni **non distruttivi** creano perlopiù disturbo o panico: possono nascondere il cursore, mostrare messaggi a schermo, reindirizzare la navigazione o inviare finte email ai contatti dell'utente facendole sembrare autentiche (come nel caso del virus Melissa).
- I danni **distruttivi** compromettono seriamente le macchine eliminando i file (come faceva il virus Jerusalem), crittografandoli tramite ransomware, modificando il registro di sistema o alterando in modo subdolo porzioni di documenti testuali.
- Infine, vi sono i danni con **intento criminale o commerciale**, focalizzati sul rubare credenziali bancarie, raccogliere dati proprietari, inviare spam o utilizzare la macchina infetta per sferrare attacchi Denial of Service (DoS) verso altri sistemi.

A livello sistemico, ripulire un computer infetto può essere un'impresa ardua. Il malware è scritto per resistere all'eradicazione, nascondendo le sue componenti in direttrici di basso livello o sostituendo file di avvio critici. Alcuni virus "multipartite" dividono il proprio codice in più porzioni: se l'utente o l'antivirus ne cancella solo una parte, le porzioni rimanenti ricostruiscono il virus originario, costringendo talvolta alla formattazione completa del disco.

2. Trasmissione e Propagazione

Per iniziare a diffondersi, il malware deve innescare l'esecuzione. La trasmissione avviene frequentemente attraverso varie tecniche:

Trasmissione tramite programmi di installazione (Setup and Installer Program Transmission)

In questo scenario, il codice virale viene inserito all'interno di un software distribuito tramite un supporto fisico o scaricato all'interno di un pacchetto di installazione. Quando l'utente esegue il programma o l'installer, il virus si installa in modo permanente sul supporto di archiviazione (come il disco rigido) o si carica nella memoria tra i programmi in esecuzione. Questo metodo richiede l'intervento umano iniziale per scaricare il file ed eseguire il programma infetto. Una volta avvenuto questo primo innesco, tuttavia, l'infezione può continuare a diffondersi senza ulteriori azioni da parte dell'utente.

Trasmissione tramite file allegato (Attached File)

Si tratta di uno dei mezzi di attivazione più comuni e fa largo affidamento sull'inganno. Il malware si nasconde in un file allegato a un messaggio (ad esempio un'email) o incorporato in un file scaricato da un sito web. L'obiettivo di chi scrive il virus è convincere la vittima ad aprire l'oggetto. Non appena il file viene aperto ed eseguito, il virus si attiva e compie il suo lavoro. È importante notare che il pericolo non si limita ai classici file eseguibili: anche file grafici, musicali o immagini possono contenere codice nascosto che viene eseguito dall'applicazione usata per visualizzarli (come nel caso dei macro virus).

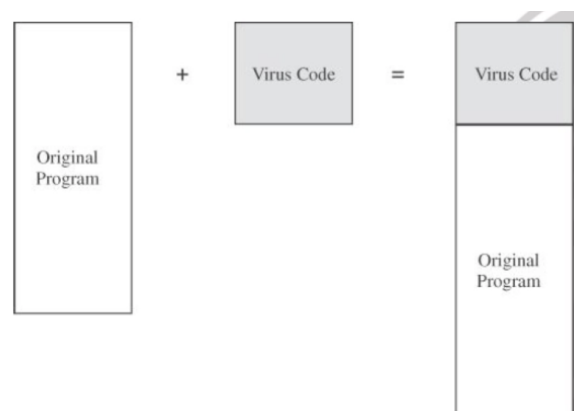
Virus nei documenti (Document o Macro Viruses)

Questa tecnica prende di mira file altamente strutturati come documenti di testo, fogli di calcolo, presentazioni, database o persino immagini. Tali documenti non contengono solamente parole o numeri, ma anche comandi legati alla formattazione, formule e collegamenti. Questi comandi fanno parte di veri e propri linguaggi di programmazione integrati che permettono l'uso di macro, variabili, procedure e chiamate di sistema. Un creatore di malware può sfruttare le funzionalità di questo linguaggio per eseguire azioni dannose, ad esempio aggiungendo il virus alle direttive di avvio dell'applicazione stessa o incorporandolo nei file di dati, in modo da infettare chiunque riceva e apra quei documenti.

Esecuzione automatica (Autorun)

Ogni sistema operativo è progettato per eseguire automaticamente determinati file o programmi di sistema e di utilità durante la fase di avvio (startup) o al momento del login dell'utente. Aggiungere un codice maligno a uno di questi file eseguiti in automatico rende il processo di attivazione e la successiva trasmissione estremamente facili ed efficaci, garantendo che il malware prenda vita ad ogni accensione della macchina.

A livello di propagazione logica all'interno dei file, un virus può posizionarsi in diverse aree. Può essere "appeso" alla fine di un programma (Appended Virus), può letteralmente "circondare" il file ospite (eseguendo azioni prima e dopo l'esecuzione del programma legittimo per modificarne l'output), oppure può integrarsi completamente, sostituendo del tutto l'effetto del target per eseguire soltanto le sue istruzioni maligne. In questo ultimo caso, l'attaccante del virus deve conoscere la precisa struttura del programma per sapere dove inserire i pezzi precisi del virus.



3. Attivazione

Affinché un malware possa causare danni, deve innanzitutto trovare un modo per farsi eseguire. Solitamente, chi scrive il codice maligno spera di sfruttare una vulnerabilità nota per la quale la vittima non ha ancora applicato una patch correttiva. I vettori storici includono vulnerabilità come i buffer overflow (sfruttati ad esempio dal worm Code Red) o richieste RPC manipolate (come nel caso di Conficker).

A livello concettuale, l'attivazione si riduce a un conflitto tra il codice malevolo, che chiameremo **Virus (V)**, e il codice legittimo che l'utente o il sistema intendevano effettivamente eseguire, il **Target (T)**. Per farsi invocare al posto del programma legittimo, il virus ha tre alternative principali:

1. **Fingersi T:** Il virus assume l'identità del target, dicendo in pratica "Io sono T".
2. **Sostituirsi a T:** Il virus spinge via il target e si mette in prima linea, dicendo al sistema "Chiama me al posto di T".
3. **Invocazione diretta:** Il virus riesce a far eseguire se stesso in modo esplicito (dicendo, in sostanza, "Eseguiami, stupido!").

A livello pratico nel file system, questa "sostituzione" avviene in due modi. Il primo metodo consiste nel **sovrascrivere fisicamente** il codice di T nello storage. Il secondo metodo, più subdolo, consiste nel **modificare i puntatori** nella directory dei file: in questo modo, quando il sistema operativo cerca il programma T, i puntatori lo reindirizzano a eseguire il codice V.

Per infettare un sistema, il malware esegue spesso un processo rapido di **One-Time Execution** (impianto iniziale). Questo avviene quando l'utente compie un'azione come scaricare un file, aprire un allegato, o subisce un download silenzioso visualizzando una pagina web. Questo primo passaggio deve essere fulmineo e invisibile all'utente.

Tuttavia, l'obiettivo dell'attaccante è spesso creare un danno continuo e ripetuto nel tempo, non un singolo assalto. Per farlo, l'infezione deve diventare parte integrante del sistema operativo ed essere ricaricata a ogni riavvio. È qui che entrano in gioco le aree più profonde del sistema.

Virus del Settore di Avvio (Boot Sector Viruses)

La sequenza di avvio (bootstrap) è un bersaglio primario. Il virus può aggiungere il proprio codice alla lista dei moduli da caricare all'accensione, oppure sostituire un modulo di avvio legittimo cambiandone l'indirizzo di caricamento o il nome. Questa tecnica è devastante per due motivi:

- **Tempismo:** Il virus ottiene il controllo *prima* che la maggior parte degli antivirus e degli strumenti di rilevamento sia attiva.
- **Invisibilità:** I file dell'area di boot sono cruciali per il sistema operativo, che di default li nasconde e ne impedisce la cancellazione per proteggerli. Il virus si nasconde quindi "dietro" le protezioni stesse del sistema.

Virus Residenti in Memoria (Memory-Resident Viruses)

Un'altra tecnica per assicurarsi un'esecuzione costante è infettare il "codice residente", ovvero le routine del sistema operativo o dei programmi usati di frequente che rimangono costantemente attive in memoria (come la routine che interpreta i tasti premuti sulla tastiera o i gestori di errori). Poiché queste routine "Terminate and Stay Resident" vengono invocate in continuazione, ogni volta che il sistema le chiama, esegue involontariamente anche il virus. Per sopravvivere al riavvio del computer, il virus residente può inserire una chiave nel registro di sistema. In questo modo, anche se l'utente nota il processo anomalo in memoria e lo chiude, il sistema lo "resusciterà" all'avvio successivo.

L'inganno dei File Passivi: Documenti e Macro

L'attivazione non richiede necessariamente un file eseguibile tradizionale (.exe). Anche file apparentemente "passivi" come PDF, immagini o brani musicali contengono dati che devono essere interpretati da un programma (come Adobe Reader). Se è presente una falla nell'interprete del software, la semplice apertura di un documento infetto può forzare il download e l'esecuzione di codice maligno.

Un caso eclatante è quello dei **Macro Virus**. Programmi come Word o Excel permettono di creare "macro", ossia script che automatizzano comandi complessi. Un virus può sfruttare la funzione delle

macro di avvio (che partono all'apertura del programma) per aggiungere le proprie direttive maligne all'applicazione e incorporarsi in ogni nuovo documento salvato, diffondendosi così a tutti i contatti a cui la vittima invierà i propri file.

4. Stealth (Invisibilità ed Evasione)

Il malware deve sfuggire al rilevamento nascondendo la sua presenza durante l'installazione, l'esecuzione e lo stazionamento sul disco. Da un punto di vista puramente informatico, identificare l'equivalenza tra due programmi (uno legittimo e uno maligno) è un problema indecidibile. Poiché gli antivirus ricercano i virus scansionando "firme" statiche (sequenze di bit invariate), gli sviluppatori di malware applicano tecniche per eluderne la rilevazione.

Per variare l'aspetto del codice nello storage vengono impiegati metodi come l'inserimento di codice e istruzioni inutili, riordino dei moduli, e steganografia (ovvero il nascondere istruzioni malevole all'interno dei pixel di immagini ad alta risoluzione senza che l'occhio umano percepisca difetti). L'evoluzione di queste tecniche porta ai *Virus Polimorfici*, capaci di cambiare il proprio aspetto in centinaia di forme posizionando casualmente il codice al loro interno. Alcuni arrivano persino a sfruttare l'autocrittografia (Encrypting viruses), mutando costantemente e lasciando in chiaro solamente la chiave e la routine necessaria all'autodecrittazione.

Contromisure e Difesa

Responsabilità dell'Utente

Un livello primario di difesa si basa sulla prudenza dell'utente: utilizzare software commerciale affidabile, isolare le macchine di test, aprire solo allegati sicuri, applicare le patch tempestivamente e usare costantemente scanner antivirus. Gli antivirus moderni hanno un alto tasso di rilevamento, eppure presentano limitazioni intrinseche dovute al fatto di essere "retrospettivi": hanno bisogno della firma del malware nel loro database. A causa di questo limite, esiste una "finestra di esposizione" chiamata *Zero-day* (dalla scoperta della falla al rilascio della patch) in cui le protezioni statiche sono inefficaci, e un utente risulta vulnerabile.

Contromisure degli Sviluppatori

Per scrivere codice sicuro, gli sviluppatori si affidano a tecniche consolidate di Ingegneria del Software. L'obiettivo è costruire un'architettura che minimizzi le probabilità di errore e isoli le eventuali falle.

Modularità (Coesione e Accoppiamento)

Un principio cardine è dividere il software in piccole unità indipendenti chiamate moduli. Un buon modulo deve rispettare quattro condizioni: **avere un unico scopo ben preciso, essere abbastanza piccolo da poter essere compreso interamente da una mente umana, essere semplice nella sua logica e agire in modo indipendente dagli altri moduli**. In gergo tecnico, i componenti devono avere un'**alta coesione** (tutti gli elementi al loro interno hanno un motivo logico e funzionale per stare insieme) e un **basso accoppiamento** (dipendono il meno possibile dagli altri componenti del sistema). I vantaggi per la sicurezza sono enormi: se si scopre una falla, questa rimane confinata nel suo modulo, ed è molto più facile effettuare test e manutenzione.

Incapsulamento e Information Hiding

L'incapsulamento permette di pacchettizzare le informazioni in modo da condividere con l'esterno solo il minimo indispensabile, mantenendo il minor numero di interfacce possibile. Si parla di *Information Hiding* quando un componente viene trattato come una "scatola nera" (black box): chi usa il modulo conosce solo gli input e gli output previsti, ma non il modo in cui il modulo esegue il suo compito. Questo

protegge il codice perché un attaccante, non conoscendone la struttura interna, farà molta più fatica a manipolarlo per scopi maligni.

Mutua Sospensione e Confinamento

Il principio della *Mutua Sospensione* suggerisce che i programmi debbano operare partendo dal presupposto che le altre routine del sistema possano essere difettose o malevole. Ad esempio, una funzione che ordina una lista non deve fidarsi ciecamente che il programma principale le fornisca dati validi, e allo stesso tempo il programma principale non deve fidarsi ciecamente che la funzione non alteri i dati. Entrambi devono implementare controlli rigorosi sugli input/output e sulla gestione degli errori. Il *Confinamento*, invece, limita strettamente le risorse a cui un programma ha accesso. Poiché i virus si diffondono transitivamente attraverso dati condivisi, isolare le applicazioni in compartimenti stagni impedisce all'infezione di espandersi all'intero sistema.

Semplicità e Diversità Genetica

"La complessità è spesso la nemica della sicurezza". Soluzioni semplici lasciano meno spazio per nascondere errori e sono molto più facili da revisionare. A questo si aggiunge la necessità di *Diversità Genetica*: basare un intero ecosistema su un singolo fornitore o creare un'integrazione troppo profonda tra i software (come faceva storicamente Windows legando il sistema operativo a Internet Explorer, Office e Outlook) è rischioso. Una vulnerabilità in un sottosistema integrato può far crollare tutto il resto. Introdurre variazioni e diversità (persino usando compilatori che generano codice oggetto leggermente diverso partendo dallo stesso codice sorgente) crea un "bersaglio mobile" che disorienta gli aggressori.

Metodologie e Limiti del Testing

Il testing è la fase critica in cui si verifica se i principi di sicurezza sono stati applicati correttamente.

I test non sono tutti uguali e coprono diverse fasi dello sviluppo:

- **Unit Testing:** Verifica il singolo componente in un ambiente controllato.
- **Integration Testing:** Controlla che i componenti funzionino correttamente quando uniti.
- **Function e Performance Testing:** Verificano il rispetto dei requisiti di sicurezza e di prestazione del sistema integrato.
- **Acceptance e Installation Testing:** Assicurano che il prodotto risponda ai requisiti del cliente e che non faccia "nulla di più" di quanto previsto una volta installato.
- **Regression Testing:** Si esegue dopo aver rilasciato una patch o un aggiornamento, per assicurarsi che le funzioni precedenti continuino a funzionare e che le performance non siano peggiorate.

Esistono due prospettive principali per condurre questi test: il

- **Black-box testing:** il tester inserisce input e verifica l'output senza poter vedere come è scritto il codice interno.
- **Clear-box testing:** il tester ha pieno accesso al codice sorgente e può generare casi di test mirati, ad esempio per controllare specifici cicli o istruzioni condizionali.

Perché il testing sia efficace, richiede ingredienti chiave come: un'elevata expertise sia sul prodotto che sul dominio logico in cui opera, un'analisi del rischio per dare priorità alle parti più critiche (rispondendo alla domanda "cosa può andare gravemente storto?"), e una continua variazione delle routine di test, poiché il testing non è una semplice "spunta" su una lista di controllo.

Contromisure specifiche per la Sicurezza

Oltre alle buone pratiche generali di ingegneria del software, per difendersi efficacemente dai malware è necessario adottare metodologie specifiche per la sicurezza. Questo richiede di cambiare mentalità, progettando l'architettura attorno a dei pilastri ben definiti e applicando tecniche di difesa attiva.

I Principi di Progettazione Sicura

Per costruire un sistema robusto, gli sviluppatori devono aderire a principi fondamentali di sicurezza fin dalla fase di design:

- **Privilegio Minimo (Least Privilege):** Ogni utente e ogni programma deve operare con i minimi privilegi possibili per portare a termine il proprio compito. Questo minimizza l'impatto di un attacco o di un errore accidentale.
- **Economia del Meccanismo:** I sistemi di protezione devono essere piccoli, semplici e diretti. Solo un sistema semplice può essere analizzato a fondo e testato in modo esaustivo.
- **Design Aperto (Open Design):** La sicurezza non deve basarsi sull'ignoranza dell'attaccante. Un meccanismo pubblico e aperto permette uno scrutinio indipendente, mantenendo segreti solo pochissimi elementi chiave (come le password o le chiavi crittografiche).
- **Mediazione Completa:** Ogni singolo tentativo di accesso deve essere verificato. Il sistema deve essere posizionato in modo che sia impossibile per un attaccante aggirare il meccanismo di controllo.
- **Basato sui Permessi (Permission Based):** La condizione di *default* deve essere sempre il blocco dell'accesso (deny). Il progettista deve identificare esplicitamente ciò che può essere accessibile, anziché fare una lista di ciò che non lo è.
- **Separazione dei Privilegi:** L'accesso a oggetti critici dovrebbe dipendere da più di una condizione (es. autenticazione dell'utente *più* una chiave crittografica). In questo modo, aggirare un solo controllo non garantisce l'accesso totale.
- **Meccanismo Meno Comune (Least common mechanism):** Bisogna ridurre al minimo la condivisione di risorse, poiché gli oggetti condivisi creano canali potenziali per la fuga di informazioni.
- **Facilità d'Uso:** Se un meccanismo di sicurezza è troppo complesso da utilizzare, è molto probabile che gli utenti cerchino dei modi per aggirarlo.

Tecniche Avanzate di Difesa e Verifica

Per testare e rafforzare queste fondamenta, si ricorre a tecniche più evolute del normale testing:

Penetration Testing (Ethical Hacking)

Consiste nell'ingaggiare un team di esperti (Tiger Team) affinché cerchino di "craccare" il sistema. Questo metodo è utilissimo per trovare falle sfuggite ai test standard. Tuttavia, richiede molto tempo (mesi) e ha un limite logico: se il sistema supera il test, non significa che sia infallibile, ma semplicemente che non soffre delle specifiche vulnerabilità testate dal team in quel momento.

Prove di Correttezza del Programma (Proofs of Program Correctness)

Per superare i limiti empirici dei test, si cerca di dimostrare in modo formale (matematico-logico) che un programma sia corretto. Ogni riga di codice viene tradotta in un'implicazione logica per dimostrare che, partendo da determinati input, il risultato finale sarà sempre quello atteso. È una pratica complessa e ha il difetto che spesso ci si concentra troppo sul "formalismo matematico", perdendo di vista le reali proprietà di sicurezza del software.

Validazione e Programmazione Difensiva

- **Validazione:** Mentre la verifica controlla la *qualità* dell'implementazione, la validazione si assicura che il sistema faccia esattamente ciò che i requisiti specificano, e *niente di più*.
- **Programmazione Difensiva:** Il difensore deve parare *tutti* i possibili attacchi, mentre all'attaccante basta trovare *una* singola falla. Per questo, il programmatore deve scrivere codice non solo corretto, ma deve anticipare "cosa potrebbe andare storto" (es. inserimento di dati errati o malformati). Un software davvero sicuro deve continuare a operare correttamente anche mentre si trova sotto attacco.